

管理者用お知らせ画面への削除機能の実装

今回の実装では、渡された一意な識別子（ `announcementId` ）を元に、データベースから対象のレコードを完全に消去するトラフィックを構築します。

Service層と**Controller層**に記述を追加することで、これを実現することが出来ます。

バックエンド: Service層の修正

`AnnouncementService.java` に、指定されたIDのお知らせを削除するメソッド（`deleteAnnouncement`）を新設します。JPAが提供する `deleteById` メソッドを利用することで、非常にシンプルに記述できます。

```
// --- AnnouncementService.java の既存のメソッドの下に追加 ---

/**
 * ロジック3: 指定されたお知らせをデータベースから完全に削除する
 * @param id 削除対象のお知らせID (UUID)
 */
public void deleteAnnouncement(String id) {
    // 安全ガード: 対象のデータが本当に存在するかチェック
    if (!announcementRepository.existsById(id)) {
        throw new RuntimeException("削除しようとしたお知らせは見つかりませんでした。ID: " + id);
    }

    // データの削除を実行
    announcementRepository.deleteById(id);
    System.out.println("=== [Service] お知らせをデータベースから削除しました (ID: " + id + ") ===");
}
```

バックエンド: Controller層の修正

`AdminAnnouncementController.java` に、フロントエンドからの削除リクエスト（HTTPの `DELETE` メソッド、または `POST` メソッドでの受付）を受け取る窓口を新設します。パスパラメータにIDを含める形式（`DELETE /api/admin/announcements/{id}`）で実装します。これはRESTfulな設計として一般的なものです。

```
// --- AdminAnnouncementController.java の既存のメソッドの下に追加 ---

/**
 * 窓口3: 指定されたお知らせの削除リクエストを受け付ける
 * DELETE http://localhost:8080/api/admin/announcements/delete/{announcementId}
 */
@DeleteMapping("/delete/{id}")
public ResponseEntity<String> deleteAnnouncement(@PathVariable String id) {
    // バックエンド側での簡易ガード（ID空チェック）
    if (id == null || id.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("【却下】 不正な呼び出しです。削除対象のIDが指定されていません。");
    }

    try {
        // サービス層の削除処理を呼び出す
        announcementService.deleteAnnouncement(id);
        return ResponseEntity.ok("お知らせを正常に削除しました。");
    }
}
```

```
    } catch (RuntimeException e) {  
        // Service側でデータが見つからなかった場合の個別エラー返却  
        return ResponseEntity.status(404).body(e.getMessage());  
    } catch (Exception e) {  
        return ResponseEntity.internalServerError().body("お知らせの削除処理中にエラーが発生しました: " + e.getMessage());  
    }  
}
```

バックエンド側の連動仕様解説

1. @DeleteMapping の採用

登録・編集時の送信メソッドである `@PostMapping` と明確に区別し、データの「破壊・消去」を表すための適切なHTTPメソッドとして `@DeleteMapping` を採用しています。

2. @PathVariable による安全なID受け取り

URLの末尾に埋め込まれたUUID（例: `/delete/abc-123-...`）を自動的に検知して、引数の `String id` に格納します。リクエストボディ（JSON）をわざわざ構築してやり取りする必要がないため、通信効率が高く軽量の処理になります。

フロントエンド側の実装

管理者が操作する `AdminAnnouncementPanel.vue` の各お知らせカード内に「削除する」ボタンを配置し、誤操作を防ぐための確認アラート（ダイアログ）を挟んでからバックエンドの削除APIと通信させるロジックを追加します。

`AdminAnnouncementPanel.vue` の修正コード

① `<template>` 部分の修正（削除ボタンの追加）

各お知らせカードのフッター領域（ `class="card-footer"` ）にある「編集する」ボタンの隣に、「削除する」ボタンを配置します。

```
<div class="card-footer">
  <small class="id-text">ID: {{ item.announcementId }}</small>

  <div class="card-actions-gap">
    <button @click="startEdit(item)" class="btn-edit-trigger">編集する</button>
```

```
<button @click="handleDelete(item.announcementId, item.announcementTitle)" class="btn-delete-trigger">
  削除する
</button>
</div>
</div>
```

② <script setup> 部分の修正（削除用通信関数の追加）

バックエンドへ DELETE リクエストを送信する関数 `handleDelete` を追加します。

```
// --- AdminAnnouncementPanel.vue の script setup 内の末尾などに追加 ---

// 【新規追加】お知らせの削除リクエストを処理する関数
const handleDelete = async (announcementId, announcementTitle) => {
  // 管理者への最終確認（誤操作ガード）
  if (!confirm(`【確認】 \nお知らせ「${announcementTitle}」を完全に削除してもよろしいですか？\nこの操作は取り消せません。`)) {
    return;
  }

  try {
    // バックエンドの @DeleteMapping("/delete/{id}") へ接続
    // パスパラメータとしてURLの末尾に直接IDを結合して送信します
    const response = await apiClient.delete(`/admin/announcements/delete/${announcementId}`);
```

```
    alert(response.data); // 「お知らせを正常に削除しました。」を表示
    // 現在編集中的数据が、たった今削除したデータだった場合はフォームを閉じる安全策
    if (form.value.announcementId === announcementId) {
        cancelEdit();
    }

    fetchAnnouncements(); // 削除後の最新リストを再取得して画面をパッと更新！
} catch (error) {
    console.error('お知らせ削除エラー:', error);
    alert(error.response?.data || 'お知らせの削除に失敗しました。');
}
};
```

③ <style scoped> 部分の修正（削除ボタン用CSS）

並び順を整えるためのGap（隙間）設定と、削除ボタンに適した赤ベースのスタイリングを追加します。

```
/* --- AdminAnnouncementPanel.vue の style scoped 内の末尾に追加 --- */
```

```
/* ボタンを綺麗に横並びにする */
.card-actions-gap {
  display: flex;
  gap: 8px;
}

/* 削除ボタンのスタイリング */
.btn-delete-trigger {
  background: transparent;
  border: 1px solid #e53935;
  color: #e53935;
  padding: 4px 12px;
  border-radius: 4px;
  font-size: 0.85rem;
  font-weight: bold;
  cursor: pointer;
  transition: all 0.2s;
}
.btn-delete-trigger:hover {
  background-color: #ffebee;
}
```

削除機能の連動テスト手順

ファイルを保存してブラウザでコントロールパネルの「お知らせ管理」タブを開き、以下のトラフィックが正常に流れるかテストしてみましょう。

- **削除のキャンセルテスト:**

- どのお知らせでも構いませんので、「削除する」ボタンを押します。
- ブラウザの確認ダイアログが表示されたら、一度「キャンセル」を押します。
- リストからカードが消えず、何も変化が起きないことを確認します（安全ガードの確認）。

- **完全削除の実行テスト:**

- 再度「削除する」ボタンを押し、今度は「OK」を押します。
- バックエンドのデータベースにDELETEリクエストが到達し、「お知らせを正常に削除しました。」というアラートが表示されるか確認します。
- アラートを閉じると同時に、**一覧から対象のカードがシュッと消え去り、最新の件数に更新されれば成功です。**